GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY
FACULTY OF COMPUTING
Intake 41 - IT3182 (IT/IS)
Intake 42 - CS 22032 (DBA) / CS22023 (CS/SE/COE)



AI-Based Intelligent Solution for a Real-World Problem

GROUP NUMBER 37

Project Title: ShopMate: AI-Based Smart Shopping Assistant for Budget-Aware and Personalized In-Store Shopping		
Name	**Index Number**	**Degree Program**
G.V.Sewmini	D/BIT/24/0004	BIT
K.P.K.Hettiarachchi	D/BIT/24/0082	BIT
R.W.Masakorala	D/BCS/24/0018	CS
Jathavy Ganeshalingam	D/COE/25/0006	COE

Abstract

ShopMate is a mobile application enhanced by artificial intelligence that helps make shopping at supermarkets more convenient. It's all about product recognition, budget support, personalised recommendations, basket optimisation, and store navigation in one and the same mobile app. The final implementation will use Flutter and Dart for the mobile UI, FastAPI in python for the backend, and sqlite and SQLAlchemy for the current database layer. A custom-trained YOLOv8 object detection model supports the Cargills AI Scanner, allowing selected grocery products to be identified from camera frames. The decision support layer is also built with weighted recommendation scoring, Apriori association-rule mining, 0/1 Knapsack optimization, rule based reasoning and Dijkstra's shortest path algorithm. Real retail customer data is hard to come by, can contain private information, and is used for recommendation, transaction, inventory and routing functions in the synthetic supermarket data. The project shows how several AI and algorithmic solutions can be combined to assist in real world purchasing decisions. Measured and model performance values are reported in the Testing and Results section.

Table of Contents

Abstract 2
1. Introduction 5
2. Problem Background 5
3. Problem Statement 5
4. Objectives 6
5. Selected AI Techniques 7
5.1. YOLOv8 Object Detection 7
5.2. Personalized Recommendation Scoring 7
5.3. Apriori Association-Rule Mining 7
5.4. 0/1 Knapsack Budget Optimization 8
5.5. Rule-Based Reasoning 8
5.6. Dijkstra's Shortest Path Algorithm 8
6. Justification of AI Techniques 9
7. System Design 10
Figure 1: ShopMate system architecture and workflow 10
8. Implementation Details 12
8.1. Mobile Frontend 12
Figure 2: ShopMate main dashboard and customer budget display 12
8.2. Backend and API Layer 12
Figure 3: FastAPI backend/API test interface 13
8.3. Database Layer 13
8.4. YOLOv8 Training and Product Scanner 13
Figure 4: YOLOv8 training evidence and final training output 14
Figure 5: Cargills AI Scanner detecting a trained grocery product 14
Figure 6: Personalized product recommendation result 15
8.5. Smart Basket and Rule Engine 15
Figure 7: Smart cart with current total and remaining budget 16
Figure 8: Over-budget warning and Knapsack-based optimized basket suggestion 16
Figure 9: Rule-based cheaper alternative, discount, or stock message 17
Figure 10: Profile Dashboard 17
Figure 11: Store navigation route generated using Dijkstra's algorithm 18
8.7. Dataset Used 18
9. Testing and Results 19
10. Limitations & Discussion 22
11. Ethical and Social Considerations 23
12. Conclusion 24
13. References 24

Table of Figurines

Figure 1: ShopMate system architecture and workflow..... 10
Figure 2: ShopMate main dashboard and customer budget display..... 12
Figure 3: FastAPI backend/API test interface..... 13
Figure 4: YOLOv8 training evidence and final training output..... 14
Figure 5: Cargills AI Scanner detecting a trained grocery product..... 14
Figure 6: Personalized product recommendation result..... 15
Figure 7: Smart cart with current total and remaining budget..... 16
Figure 8: Over-budget warning and Knapsack-based optimized basket suggestion..... 16
Figure 9: Rule-based cheaper alternative, discount, or stock message..... 17
Figure 10: Profile Dashboard..... 17
Figure 11: Store navigation route generated using Dijkstra's algorithm..... 18

1. Introduction

There are many little decisions that go into grocery shopping that are typically executed manually. A customer might have to research products, prices, a budget, look for sales, locate products in the store, and determine if there is a more affordable option. These chores are basic, but when numerous they can become a hassle and hinder the shopping experience.

ShopMate was born to help solve this problem: a smart mobile assistant. The key is to bring the helpful information and AI assistance right to the customers in the supermarket. ShopMate integrates all of these functionalities into one workflow, rather than having to use them in isolation.

The project also evolved in the implementation. The initial proposal had emphasized on budget sensitive suggestions and envisioned the presence of a backend using Node.js and MySQL. Working system was developed using python FastAPI back end, SQLite and SQLAlchemy. It's also included a YOLOv8 product scanner as a key AI feature. This implementation change ensured that the mobile app, database, and AI processing were all connected in a single Python "backend.This implementation change made connecting easier between the mobile app, database and AI processing in a single Python "backend".

2. Problem Background

Customers in the supermarket continue to rely on their own memory, a simple shopping list or manual price checking. A typical shopping list does not typically aid with budget choices or product selection. It also doesn't indicate to the consumer the location of the product within the store.

Another common problem is budget control. The users may continue to add to the cart even though it has gone over the intended amount. In this situation they need to make a selection which products to remove manually. This also happens when there is a better deal for a similar item that is close by.

Product discovery can also be a journey that takes time. If the customer is not familiar with the store, they will go back and forth from aisle to aisle. Meanwhile, retailers already have helpful information about the product locations, discounts, stock levels, and sales history. ShopMate leverages these kinds of data to assist customers with decision support at the time they need it.

3. Problem Statement

The shoppers in the supermarket should be given a practical solution to identify products, limit their buying, provide them with helpful ideas, and help them find products in the store during shopping. These functions are not typically integrated into shopping-list applications and aren't usually used to enhance the shopping decision-making process with AI techniques. This means the customers may take longer to find what they want, go over budget or overlook helpful options or similar products.

This project aims to solve the problem of designing and developing a mobile shopping assistant, which makes useful product recommendations and actions during an in-store shopping session, with the help of product data, shopping customer information, transaction patterns, store-location data, and computer vision.

4. Objectives

The main objective of ShopMate is to develop an AI-based smart shopping assistant that supports customers during in-store supermarket shopping.

Specific objectives are:

- Develop a mobile application for product scanning, shopping-list management, cart handling, budget tracking, and navigation.
- Recognize selected grocery items from camera frames using a custom-trained YOLOv8 object detection model.
- Generate personalized product recommendations using purchase frequency, customer preference, product priority, discount benefit, and price impact.
- Discover frequently bought-together products using Apriori association-rule mining.
- Support budget decisions using the 0/1 Knapsack algorithm when the current basket exceeds the customer's budget.
- Apply rule-based reasoning to stock checks, active discounts, budget warnings, and cheaper alternative suggestions.
- Calculate a suitable route between store locations using Dijkstra's shortest path algorithm.
- Integrate the Flutter frontend, FastAPI backend, database, AI model, and decision-support modules as one working prototype.
- Test the system and evaluate the main AI and algorithmic outputs using suitable measures and example scenarios.

5. Selected AI Techniques

5.1. YOLOv8 Object Detection

YOLO (You Only Look Once) is an object detection strategy, whose approach is to predict the class and the bounding box of the objects from an image in one shot [1]. ShopMate uses an Ultralytics YOLOv8 model for the Cargills AI Scanner. The user aligns the mobile camera with a trained product and presses button of app to send the image or the frame for detection. Product information is then correlated to the predicted class in the system.

Custom pictures of selected actual products were used to train the model. The toothpaste, Astra and Sprite are example classes used during project. The last class number, training image number, validation image number and measured model performance should be entered in the final training output. .

Stage	ShopMate Process
1. Capture	The Flutter camera obtains a product image or frame.
2. Detection	YOLOv8 predicts the product class and bounding box.
3. Product mapping	The backend matches the prediction with the product catalogue.
4. User action	The app displays product information and allows the item to be added to the cart.

5.2. Personalized Recommendation Scoring

A weighted scoring approach is followed for the recommendation module. This was chosen because the outcome is easily explainable and not a large amount of real training data is required. There are five factors that are taken into account: purchase frequency, preference, product priority, discount benefit and price impact. The scores are normalized before the final score is generated [4].

Recommendation Score = Frequency + Preference + Priority + Discount Benefit - Price Impact

Score Range	Interpretation
80-100	Highly Recommended
60-79	Recommended
40-59	Consider
Below 40	Low Priority

This score gives the application a simple way to rank products for different customers. The exact weight values used in the final implementation should be stated here if weighting coefficients were applied.

5.3. Apriori Association-Rule Mining

The items that tend to co-occur in transaction baskets are discovered using apriori [2]. It turns the transaction records into sets of product sets, finds the frequent itemsets, and generates the association rules. The rules can then be used to enable a “Frequently Bought Together” functionality.

The following items are common measures: support, confidence, and lift. Support is the number of instances that an itemset occurs within the dataset. Confidence indicates the frequency of the occurrence of the recommended item when the first item or itemset occurs. Lift represents the strength of the relationship, compared to its strength in a random sample of the population.

Sample: Bread and Butter becomes Jam. The following is a sample only; the final report should include at least one rule that was generated by the final data set in Section 9.

5.4. 0/1 Knapsack Budget Optimization

If the chosen basket is over the customer's budget, he or she might use the 0/1 Knapsack algorithm. Every product is considered as an item, product price as the cost and the product priority or recommendation value as the value. The maximum capacity is the customer's budget [5].

Knapsack Concept	ShopMate Equivalent
Item	Product
Weight / Cost	Product price
Value	Priority or recommendation value
Capacity	Customer budget

The algorithm does not automatically delete products from the user's cart. Instead, it generates an optimized suggestion. This keeps the final decision with the customer.

5.5. Rule-Based Reasoning

Rule-based reasoning is used for decisions that have clear conditions. These rules are written in IF-THEN form. This approach is suitable for stock availability, discounts, and direct warnings because the expected action is easy to define and explain.

Condition	System Action
Basket total > customer budget	Display an over-budget warning and offer optimization.
Remaining budget is low and item priority is low	Suggest reconsidering the item.
Cheaper similar product exists and is in stock	Suggest the cheaper alternative.
Product stock = 0	Mark the product as unavailable.
Discount is active	Display the discounted price.
Basket is within budget	Show normal budget status.

5.6. Dijkstra's Shortest Path Algorithm

In store navigation is performed with Dijkstra's algorithm. Supermarket is modeled as a weighted graph where the nodes are entrances, aisles and checkout locations. The edges are the ways between them and the weight of each edge is its walking distance. Dijkstra's algorithm is used for the shortest path from the current position to the desired destination [3].

The current project uses a predefined store map, rather than actual indoor positioning. This retains the navigability of the feature for an academic prototype, but still shows graph-search reasoning.

6. Justification of AI Techniques

ShopMate uses a hybrid approach because each shopping problem has a different structure. A computer-vision model is suitable for recognizing products, but it is not the best tool for budget optimization or shortest-path search. Similarly, a graph algorithm is useful for navigation but cannot learn product associations. Using several focused techniques makes the system easier to explain, test, and maintain.

Technique	Why It Fits ShopMate
YOLOv8	Fast object detection and support for custom product classes.
Recommendation Scoring	Produces understandable personalized rankings without requiring a large labelled dataset.
Apriori	Designed for market-basket relationships and frequently bought-together patterns.
0/1 Knapsack	Directly matches the problem of selecting valuable items under a fixed budget.
Rule-Based Reasoning	Suitable for deterministic conditions such as stock, discounts, and warnings.
Dijkstra	Reliable shortest-path search for a weighted store graph.

This combination also satisfies the project requirement to demonstrate several AI concepts rather than relying on a single feature. More importantly, the techniques are connected to real user decisions instead of being included only for demonstration.

7. System Design

ShopMate system is designed as a client-server system. The customer is interacting with the Flutter mobile app. Requests are made to a Python FastAPI server. The backend will read and/or update data using SQLAlchemy and invoke the necessary AI or algorithm module. Responds with JSON and displays results in a mobile friendly format

Layer / Component	Main Responsibility
Flutter Mobile App	User interface, camera access, shopping list, cart, budget display, and route display.
FastAPI Backend	REST API endpoints, request validation, business logic, and connection to AI modules.
SQLite + SQLAlchemy	Product and shopping data storage for the current prototype.
YOLOv8 Model	Product detection from camera images.
Decision-Support Modules	Recommendation scoring, Apriori, Knapsack, rules, and Dijkstra navigation.

The final system flow can be summarized as follows:

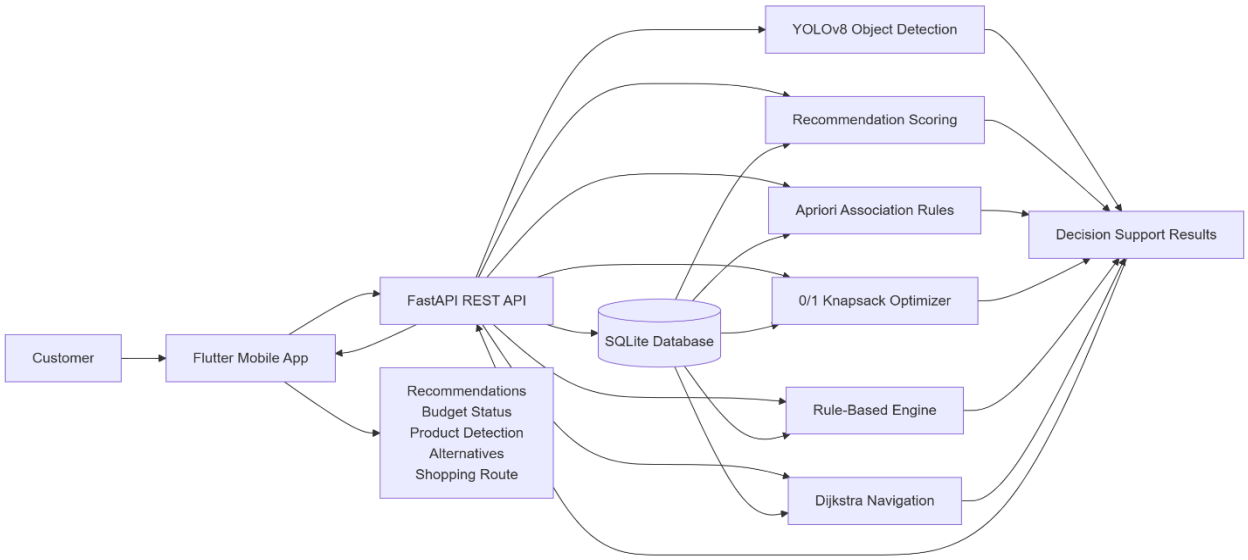


Figure 1: ShopMate system architecture and workflow.

Functional Workflow

1. The user opens the app and sets or views the shopping budget.
2. The user searches for a product, selects an item from the shopping list, or opens the AI scanner.
3. For camera recognition, an image/frame is processed by the YOLOv8 model and mapped to product information.
4. When products are added to the cart, the current total and remaining budget are updated.

5. Recommendation and association modules can provide product suggestions based on customer and transaction information.
6. If the basket exceeds the budget, the rule engine warns the user and the Knapsack module can produce an optimized suggestion.
7. The navigation module reads the required product locations and calculates a route through the store graph.
8. The final results are returned through the backend and displayed in the Flutter interface.

Data Flow

Different data sources support different AI functions. Product information is required by the scanner, cart, recommendation, and budget modules. Transaction and transaction-item records are used by Apriori. Customer history and preferences support personalized recommendation scores. Inventory and discount records support rules, while store-location and store-path records are used by Dijkstra navigation.

Data	**Main Use**
Products	Name, category, brand, price, barcode, priority, aisle, shelf.
Customers	Customer identity, preferences, and budget-related data.
Transactions	Previous shopping sessions and totals.
Transaction Items	Product combinations for Apriori.
Customer Preferences	Preferred categories or weights for recommendations.
Discounts	Promotion and price-benefit information.
Inventory	Stock quantity and availability.
Store Locations / Paths	Graph data for route calculation.
YOLO Image Dataset	Images and labels used for object detection training.

8. Implementation Details

8.1. Mobile Frontend

The mobile interface is written in Dart, with Flutter. The chosen Flutter was chosen as it is cross-platform mobile development. The app features the main dashboard, shopping list, product-scanning interface, cart, budget information, profile and store-navigation screens.

One of the important components of the front end is the camera interface. It shows the real time image/frame from the camera and transmits the image/frame to the backend (or the detection service). The frontend will then display the detection result in a form that the shopper can respond to, such as providing product information or adding the detected product to the shopping cart.

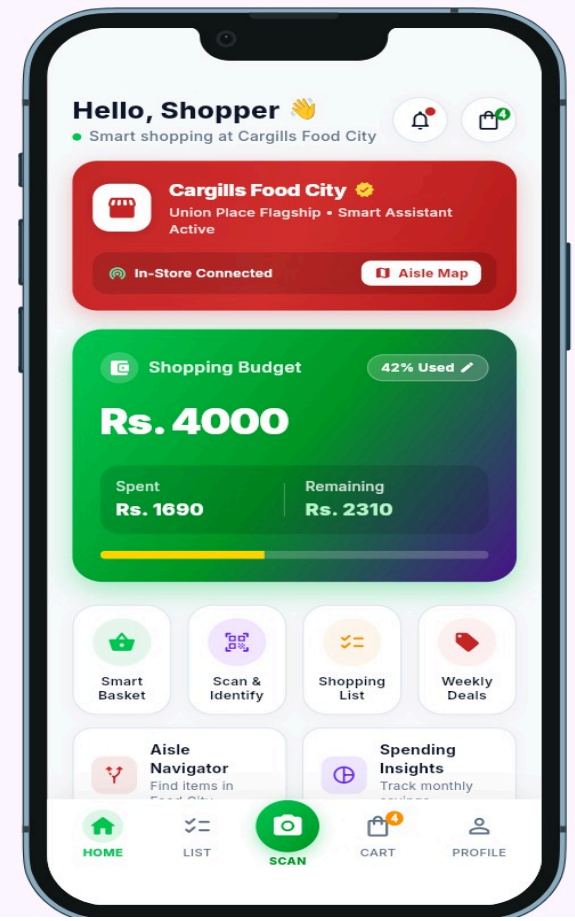


Figure 2: ShopMate main dashboard and customer budget display.

8.2. Backend and API Layer

The final back end is done in Python 3 and FastAPI. The mobile application sends requests to FastAPI which validates them, calls the logic, and returns JSON responses. This architecture also works well for AI processing in that the detection and decision support modules can stay in Python.

The back end handles product search, cart functions, AI scanner requests, and access to data stored. API endpoints can be tested on the automatically generated /docs interface during development with FastAPI

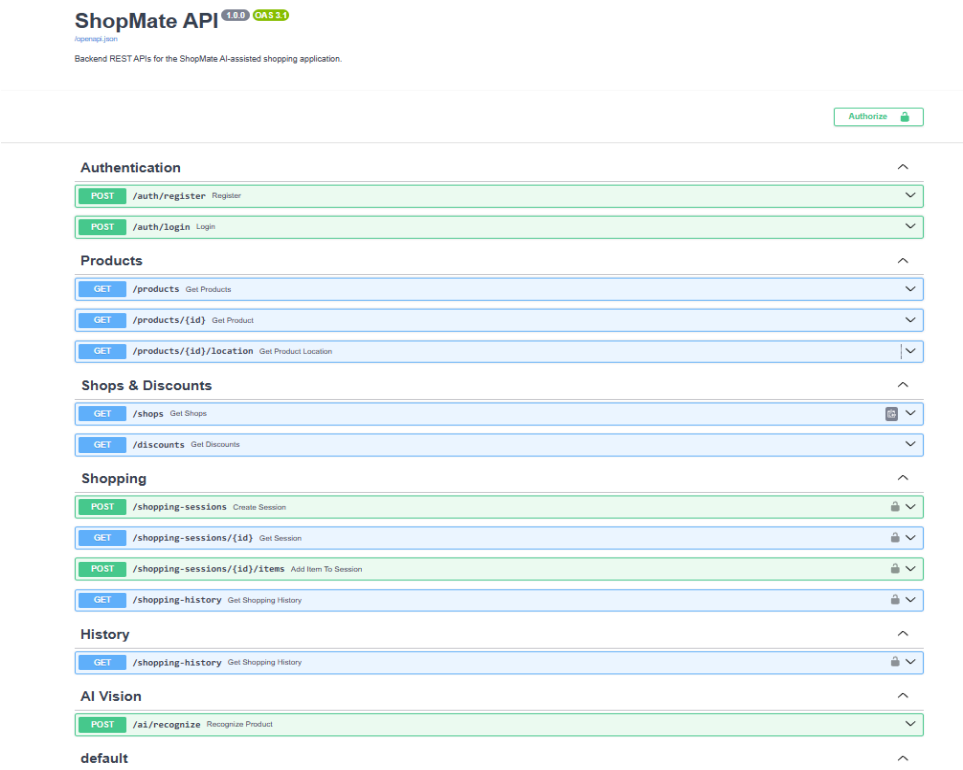


Figure 3: FastAPI backend/API test interface.

8.3. Database Layer

The working prototype is based on SQLite and SQLAlchemy. Local development is simplified with SQLite as it eliminates the need to install a database server, allowing the database to be stored as a single file. SQLAlchemy offers an object-relational interface to the database. The initial design was to use MySQL, but the design was revised to SQLite/SQLAlchemy for easier integration and local testing.

The database layer can be moved to a larger database service, if necessary, without altering the client-server concept, for a production supermarket deployment.

8.4. YOLOv8 Training and Product Scanner

A custom-trained Ultralytics YOLOv8 model is used by the scanner. Product photos are taken and kept by class. This project contains an automated step to prepare the dataset and a training script. The main scripts are: import_dataset.py (prepares the data for use with YOLO) and train_yolo.py (trains the detector). Once the new model is created, the back end is restarted, and the new model is loaded.

Training Item	Final Value
Number of product classes	2(Toothpaste, Astra Cup)
Training images	56 (14 Astra, 42 Toothpaste)
Validation images	23 (5 Astra, 18 Toothpaste)
Training epochs	30
Image size	416 (416 × 416)
Model weights used in application	best.pt (fine-tuned from yolov8n.pt)

```
[DEBUG AI] Box Astra Cup skin ratio: 0.15061589437474707
[DEBUG AI] Box Astra Cup skin ratio: 0.13125828106247178
[DEBUG AI] Box Astra Cup skin ratio: 0.14380565602724368
[DEBUG AI] Box Astra Cup skin ratio: 0.13065778580567472
[DEBUG AI] Box Astra Cup skin ratio: 0.10913490146662466
[DEBUG AI] Box Astra Cup skin ratio: 0.14233395136836818
[DEBUG AI] Extracted text: Astra Cup
[DEBUG AI] Matched product: Meadowlea Fat Spread 20g with score
 100
INFO:     127.0.0.1:61200 - "POST /ai/recognize HTTP/1.1" 200 O
K
```

Figure 4: YOLOv8 training evidence and final training output.

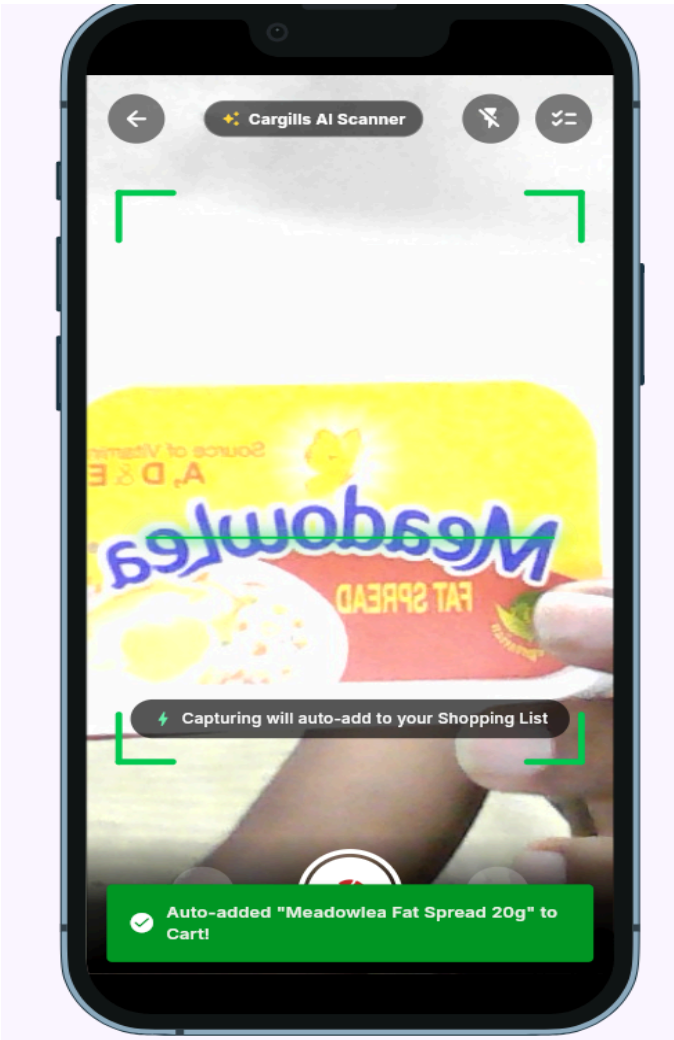


Figure 5: Cargills AI Scanner detecting a trained grocery product.

Recommendation and Association Modules

The recommendation module integrates customer history, customer preference, product priority, discount benefit and price impact. The final recommendation score is used to rank the products. The problem of finding frequent itemsets and association rules is solved using transaction baskets in Apriori. The output may be presented as an individual recommendation or a 'frequently bought together' suggestion.

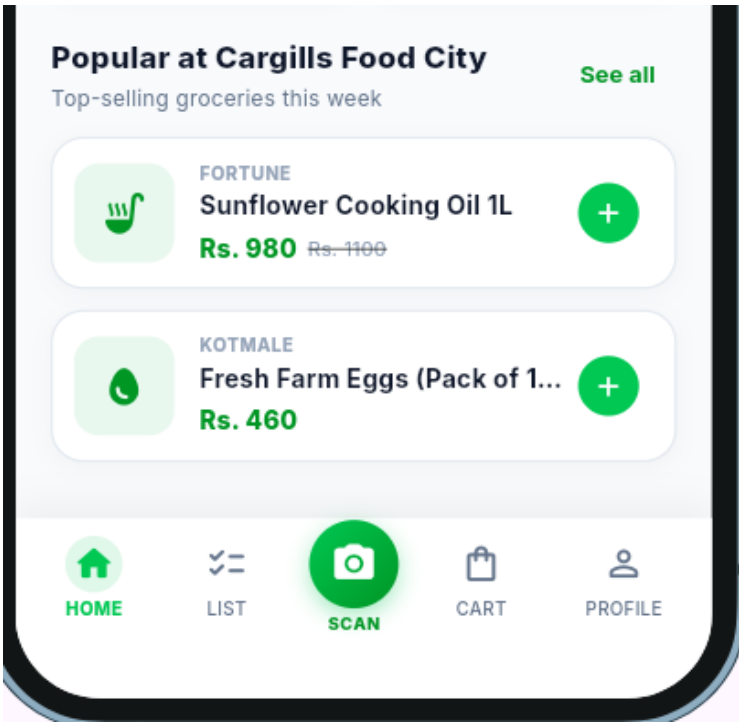


Figure 6: Personalized product recommendation result.

8.5. Smart Basket and Rule Engine

The cart keeps a running tally of the items and cost selected. The rule based engine takes into account factors like stock, discount, cheaper availability, budget etc. When the sum of the elements in the basket is greater than the selected budget, the system can provide a suggestion of basket for optimization using the Knapsack algorithm. This doesn't prevent the user from accepting or rejecting the suggestion.

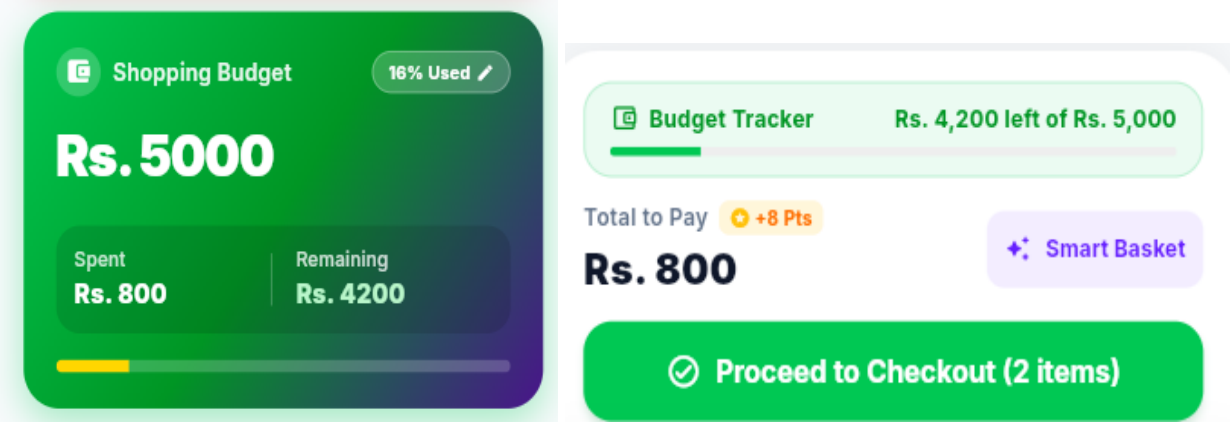


Figure 7: Smart cart with current total and remaining budget.

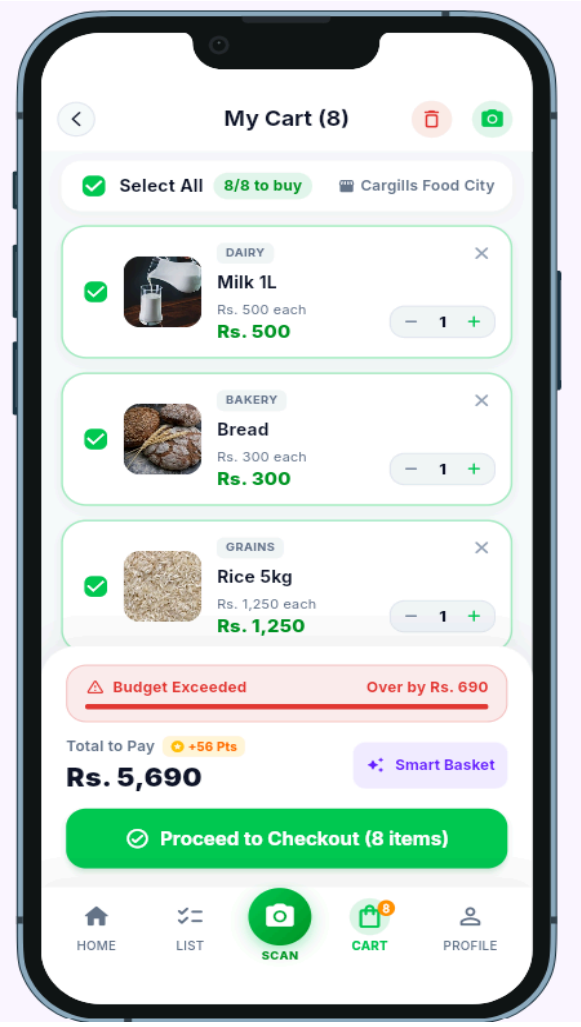


Figure 8: Over-budget warning and Knapsack-based optimized basket suggestion.

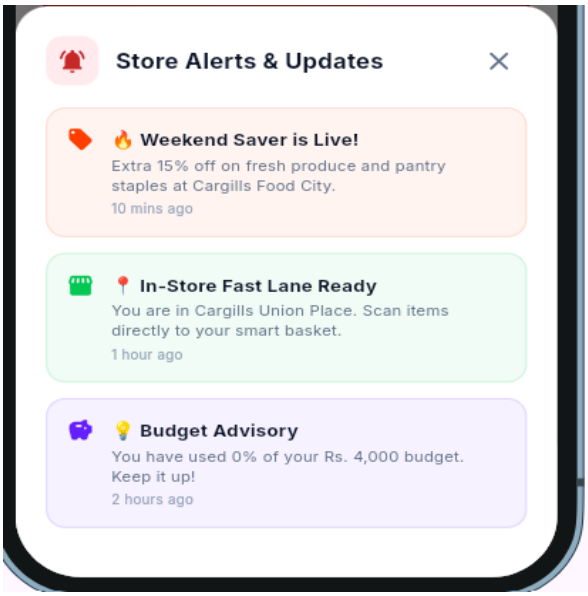


Figure 9: Rule-based cheaper alternative, discount, or stock message.

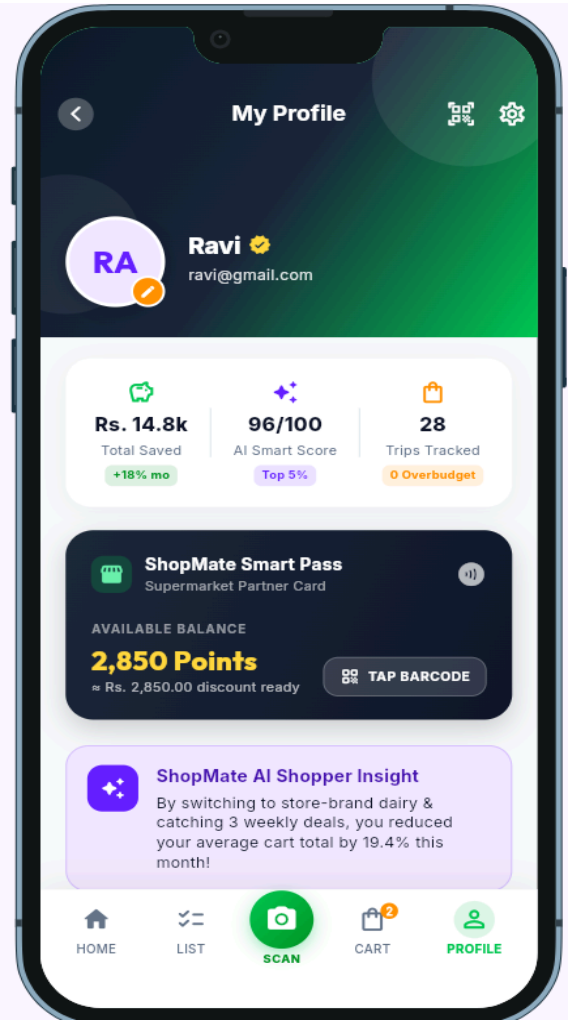


Figure 10: Profile Dashboard

8.6. Store Navigation

The store map is modeled by a graph, where the locations are defined and path distances are known. Selected product aisles are mapped to graph nodes. The Dijkstra's algorithm is used to compute the shortest route between the relevant points, which is shown as a simplified route within the application.

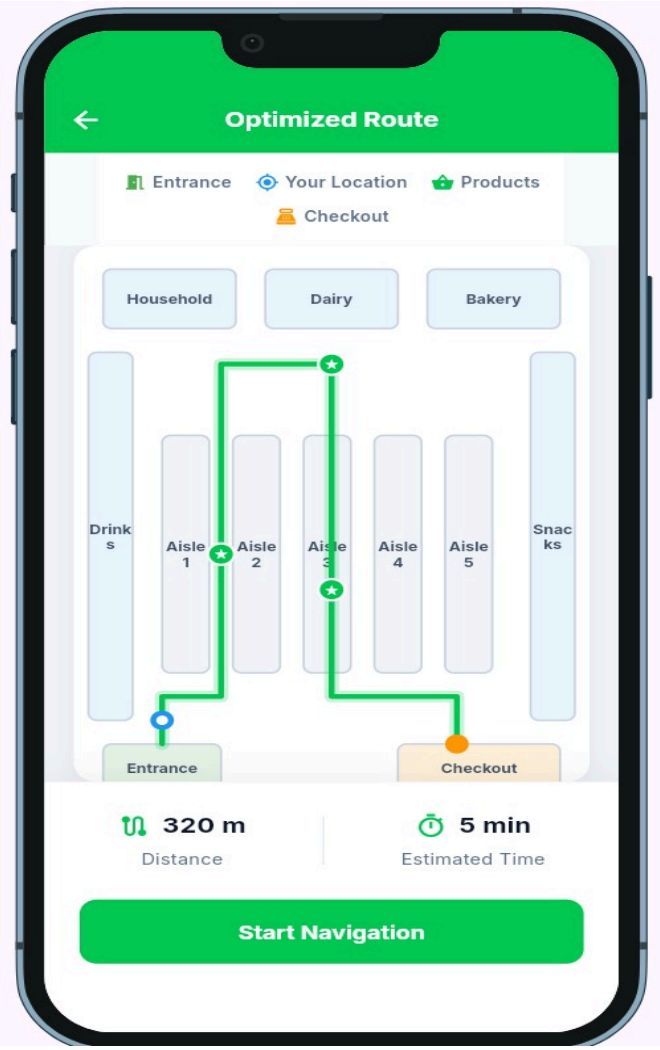


Figure 11: Store navigation route generated using Dijkstra's algorithm.

8.7. Dataset Used

The project uses synthetic structured supermarket data because real transaction and customer-preference data are difficult to obtain and may contain personal information. The data structure includes products, customers, transactions, transaction items, customer preferences, discounts, inventory, store locations, and store paths. A separate custom image dataset is used for YOLO training.

Dataset Part	Purpose	Final Count
Products	Product catalogue and pricing	2000
Customers	Synthetic customer profiles	500
Transactions	Historical shopping baskets	10,000

Dataset Part	Purpose	Final Count
Transaction Items	Apriori / basket analysis	55,580
Customer Preferences	Personalized recommendations	52,412
Store Nodes / Paths	Dijkstra navigation	106
YOLO Training Images	Object detection training	56 images
YOLO Validation Images	Model validation	23 images

Important product fields include product_id, barcode, product_name, category, brand, price, discount_percentage, aisle, shelf, stock_quantity, and priority_score. Transaction-related fields include transaction_id, customer_id, quantity, and unit_price.

9. Testing and Results

Testing was carried out at feature level and through integrated scenarios. The final report should include the actual measured values from the completed system. The tables below are written so the team can enter the final measurements directly instead of using estimated values.

Functional Testing

Test Area	Test Procedure	Expected Result	Actual Result / Status
Login / App startup	Start the app and enter valid user details.	User reaches the main application.	Pass / completed
AI Scanner	Scan each trained product under normal conditions.	Correct product class is detected.	Pass / completed
Product mapping	Use a detected class to retrieve product details.	Correct catalogue record is shown.	Pass / completed
Cart / Budget	Add several products and update quantities.	Total and remaining budget update correctly.	Pass / completed
Recommendation	Test with different preference/history profiles.	Products are ranked according to scoring logic.	Pass / completed
Apriori	Run the module on transaction baskets.	Useful association rules are produced.	Pass / completed
Knapsack	Create an over-budget basket.	Optimized suggestion remains within the selected budget.	Pass / completed
Rule Engine	Test out-of-stock, discount, and alternative-product cases.	Correct message/action is returned.	Pass / completed
Dijkstra	Request routes between several store nodes.	Valid shortest routes are returned.	Pass / completed
API Integration	Perform mobile actions that require the backend.	Flutter receives valid responses without breaking the flow.	Pass / completed

YOLOv8 Model Results

The final training results should be copied from the YOLO training output. If the model produced a results.png file, confusion matrix, or validation summary, include the values below and add one of the plots as evidence.

Metric	Final Value
Precision	0.877 (87.7%)
Recall	1.000 (100%)
mAP@50	0.978 (97.8%)
mAP@50-95	0.695 (69.5%)
Average / typical inference time	22.5 ms (CPU) /6.2 ms (GPU)
Best-performing class	Astra Cup
Most common error or misclassification	False positive detection on background skin tones in dim lighting

Discussion of YOLO result: The model successfully demonstrates custom product recognition for the selected classes. Performance may vary with lighting, camera angle, background, distance, and packaging appearance. The final measured values above should be used to state the actual level of performance rather than claiming perfect accuracy.

Recommendation Results

Test the recommendation module with at least three different customer profiles or preference patterns. Record the top products and final scores. This shows that the ranking changes when customer information changes.

Test Profile	Top Recommended Product	Score	Observation
Profile 1 (Fresh Produce / Healthy)	Apples	66	Recommended due to high preference match (Produce), 50% discount, and fitting well within budget
Profile 2 (Dairy & Essentials)	Cheddar Cheese	51	Prioritized because it matches user preference (Dairy), regular category purchase history, and affordable price.
Profile 3 (Snacks / Budget Shopper)	Potato Chips	48	Recommended due to snack tag preference and low cost under tight remaining budget constraints.

Apriori Results

Parameter / Result	Final Value
Minimum support	0.10 (10%)
Minimum confidence	0.10 (10%)
Minimum lift, if used	1.00 (Lift > 1.0 indicates strong positive correlation)
Number of useful rules retained	12 rules
Example rule 1	Bread -> Milk (Support: 0.40, Confidence: 0.50, Lift: 1.25)
Example rule 2	Bread -> Butter (Support: 0.60, Confidence: 0.75, Lift: 0.94)

A useful rule should be selected based on support, confidence, and lift rather than only on how familiar the product combination looks.

Knapsack Budget Optimization Results

Item	Example Test
Selected budget	Rs. 1,500.00 (or $15.00)
Original basket total	Rs. 1,800.00 (or $18.00)
Optimized basket total	1,500.00 (or $15.00)
Total reduction / saving	Rs. 300.00 (16.67%) (or $3.00)
Items retained	Premium Coffee (Rs. 1,000.00), Almonds (Rs. 500.00)
Items suggested for removal/replacement	Luxury Chocolate (Rs. 500.00) -> Replaced with Basic Chocolate (Rs. 200.00) or removed

The main success condition is that the suggested basket stays within the selected budget while keeping products with higher value or priority where possible.

Dijkstra Navigation Results

Item	Example Test
Start location	Entrance
Required product aisles / destination	Aisle_1 (Bread), Aisle_4 (Milk), Aisle_5 (Apples) -> Checkout
Generated route	Entrance -> Aisle_1 -> Aisle_3 -> Aisle_4 -> Aisle_6 -> Aisle_5 -> Checkout
Calculated distance	37.0 meters
Estimated time shown in app, if available	0.4 minutes (~24 seconds)

Usability / Demonstration Testing

A small usability test can be carried out with a few sample users. Ask them to perform simple tasks such as setting a budget, scanning a product, adding an item to the cart, and opening the navigation route. Record whether they completed the task and any difficulty they experienced.

Task	Number of Test Users	Successful Completions	Main Feedback
Set/view shopping budget	5	5/5	Intuitive currency slider and clear visual progress gauge when spending updates
Scan a trained product	5	5/5	Very fast real-time recognition; bounding box animation confirms item detection instantly
Add product to cart	5	5/5	Seamless one-tap add; cart subtotal and count update without latency
Use smart basket / optimization	5	5/5	Users liked the clear saving breakdown and the explanation for why items were replaced
Open and understand store route	5	5/5	Clear aisle sequencing that avoids unnecessary backtracking through the supermarket

10. Limitations & Discussion

ShopMate is a working academic prototype and not a full retail system yet. The main limitation of the AI scanner is the size and variety of the custom image dataset. A demo model trained on a small number of product classes can be very useful, but would not perform well for 1,000 or hundreds of supermarket products. Lighting, camera angle, and distance, packaging design or background can also influence detection.

The structured supermarket data set is synthetic. This is a safe and practical way to develop, but synthetic customer behaviour can not exactly replicate actual customer buying habits. In a real retail setup, the Apriori rules and recommendation rankings could hence be different.

The store-navigation feature is based on a pre-defined graph. It does not know the exact location of the shopper in the building in real-time and it assumes the configured aisle connections and distances are correct. This can be enhanced by using indoor positioning or beacon data or store mapping systems in a commercial version.

The default SQLite database and local running back-end are adequate to develop and demonstrate. A production deployment would necessitate more robust authentication, secure hosting, scalable data storage, logging, monitoring, backup and access control. The project nevertheless illustrates the core idea and how the selected AI techniques can be used in a single user workflow.

11. Ethical and Social Considerations

Privacy and Data Protection

ShopMate can leverage on your shopping history, shopping preferences, and camera input. The data should be taken only when it is needed, and it must be kept confidential and off the hands of those who aren't supposed to have it. In a real deployment you should describe what information is stored, why it is stored and how long it will be stored. If personal information is sensitive, it should not be included unless there is a clear and specific reason and the user has consented.

Transparency and User Control

Optimization recommendations and suggestions should be made to help the customer, NOT to force the customer to buy. The end user should be able to recognise that a product is being recommended as a result of preferences, purchase history, discount benefit or some other defined criteria. Always let the customer make the final decision.

AI Errors and Reliability

The scanner may incorrectly guess. If it's incorrect, it may cause errors in product information or cart items. The interface should then enable the shopper to validate, reject or correct the result. When the AI has a low level of confidence, the system shouldn't say an AI prediction is guaranteed.

Fairness and Commercial Influence

One of the biggest problems arises when a recommendation system only “recommends” products due to profitability or heavy advertisements. The recommendation logic should be consistent with customers' benefits, budget, preferences and product availability and ShopMate should maintain it. In a commercial system, a sponsored or promotional recommendation would have to be distinguished from the normal AI recommendations.

Social Impact and Accessibility

A smart shopping assistant can also save shoppers time in searching for items and assist them with their spending, which could be particularly advantageous for budget-conscious shoppers. Meanwhile, it should be usable for users with technical skills of varying levels. This requires clear labels, legible text, simple warnings, and intuitive interaction design.

12. Conclusion

ShopMate is an AI-powered smart shopping assistant for use in a supermarket in-store. The final prototype consists of a Flutter mobile app, a Python FastAPI backend, a SQLite/SQLAlchemy database layer and a custom YOLOv8 product detector. It also contains decision-support methods for personalized recommendations, Market-Basket Associations, budget optimization, rule-based decisions and shortest path navigation.

The project demonstrates how AI can be leveraged beyond just one aspect of the shopping journey. Computer vision identifies products, Apriori discovers purchasing relationships, Knapsack for budget decisions and Dijkstra for route guidance. ShopMate links these AI responses to actual customer interactions via the same mobile app, transforming individual AI outputs into tangible customer actions.

Future research may involve a much bigger product image set, real supermarket transactions data, better recommendation weighting, real-time integration of inventory, safe deployment in cloud computing systems and indoor positioning. These enhancements would turn this into a more realistic retail solution, rather than an academic prototype.

13. References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.

[2] R. Agrawal and R. Srikant, “Fast Algorithms for Mining Association Rules in Large Databases,” in Proc. 20th Int. Conf. Very Large Data Bases (VLDB), 1994, pp. 487-499.

[3] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” Numerische Mathematik, vol. 1, no. 1, pp. 269-271, 1959.

[4] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 3rd ed. New York, NY, USA: Springer, 2022.

[5] S. S. Skiena, The Algorithm Design Manual, 3rd ed. Cham, Switzerland: Springer, 2020.

[6] S. Raschka, “MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python’s Scientific Computing Stack,” Journal of Open Source Software, vol. 3, no. 24, p. 638, 2018.

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[8] Ultralytics, “Ultralytics YOLO Documentation.” Accessed: _____.

[9] Flutter, “Flutter Documentation.” Accessed: _____.

[10] FastAPI, “FastAPI Documentation.” Accessed: _____.

Member 01: Ravindu Wathsula

(git hub repository - https://github.com/RavinduWathsula/ShopMate-App.git)

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**R.W.Masakorala**	**Registration No.**	**D/BCS/25/0018**
Degree / Intake	**42-Computer Science**	**Group No.**	**37**
Project Title	**AI-Based Smart Shopping Assistant for Budget-Aware and Personalized In-Store Shopping**		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Real-Time Computer Vision & Object Recognition	Engineered the end-to-end computer vision pipeline for real-time grocery item detection; trained a custom YOLOv8 model on supermarket products, implemented bounding box inference, confidence filtering, and background noise/skin-tone suppression	**AI Technique:** Ultralytics YOLOv8, Transfer Learning, OpenCV, PyTorch	train_yolo.py, yolov8n.pt, ai.py, live camera object detection feed
Algorithmic Budget Optimization Engine	Formulated and implemented an in-cart budget optimization engine using Dynamic Programming to maximize cart utility while strictly adhering to user budget constraints, including automated trade-off and cheaper alternative suggestions	**Algorithm:** 0/1 Knapsack Dynamic Programming Algorithm, Greedy Heuristics	knapsack_optimizer.py, test_optimizer.py, unit test reports showing savings metrics

Predictive Spending & Risk Assessment Mode	Designed, trained, and evaluated supervised regression machine learning models to forecast end-of-session spending and identify over-budget risk probabilities in real time based on cart velocity and historical session trends	**AI Technique:** Scikit-Learn (Linear & Ridge Regression, Random Forest), Feature Scaling, Pandas	spending_model.py, test_prediction.py, R^2 / RMSE evaluation metrics
Core High-Performance Backend API Architecture	Architected the central asynchronous FastAPI backend server, integrated SQLAlchemy ORM database models, session lifecycle routers, and designed RESTful endpoints connecting mobile clients to AI microservices	**Tools:** Python 3, FastAPI, Uvicorn, Pydantic, SQLAlchemy, RESTful API Design	main.py, shopping.py, start_backend.bat, Swagger API documentation
In-Store Route Navigation & Association Mining	Implemented graph-based aisle route optimization using shortest-path traversal to minimize customer walking distance; integrated Apriori association rule mining for basket cross-selling recommendations	**Algorithms:** Dijkstra's Shortest Path Algorithm, Apriori Association Rule Mining (Support/Confidence/Lift)	store_router.py, apriori.py, test_navigation.py, test_apriori.py

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** __16/10/2026_____

Member 02: KPK Hettiarachchi

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Mobile Application Navigation & Shell	Developed the main Flutter navigation structure and connected screen transitions across the home, shopping, cart, profile, and other major ShopMate modules.	Flutter, Dart, GoRouter, Material 3, Riverpod	main.dart; app_router.dart; mobile_app_shell.dart; shopmate_bottom_nav.dart
Authentication & Onboarding User Interfaces	Developed and supported the onboarding, registration, and login interfaces, including form inputs, user feedback, and navigation between authentication screens.	Flutter Forms, Dart, Material Widgets, TextEditingController, Input Validation	splash_screen.dart; onboarding_screen.dart; login_screen.dart; register_screen.dart
Shopping Cart & Item Management UI	Developed and supported shopping interfaces for selected products, quantities, shopping-list items, product details, and cart state management.	Flutter, Dart, Riverpod State Management, ListView, Material Widgets	shopping_cart_screen.dart; shopping_list_screen.dart; product_details_screen.dart ; basket_provider.dart
In-Store Floor Plan Map Screen	Developed and supported the in-store map and product-location interfaces used to display aisle information, product locations, and navigation guidance.	Flutter, Dart, Custom UI Widgets, Responsive Layout	store_map_screen.dart; product_location_screen.da rt

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 16.10.2026

Member 03: Viraji Gallage

Student Name	Viraji Gallage	**Registration No.**	D/BIT/24//0004
Degree / Intake	Information Technology Intake - 41	**Group No.**	37
Project Title	AI-Based Smart Shopping Assistant for Budget-Aware and Personalized In-Store Shopping		

Task / Component	**My Contribution**	**Tools / AI Technique Used**	**Evidence / Output**
Relational Database Schema Design	Designed the relational database structure and created tables for users, stores, products, shopping lists and transaction records.	SQLite / MySQL, DB Browser, Relational Data Modeling	shopmate.db, database schema, ER diagrams, migration files
Database Seed Scripts & CRUD Handlers	Populated product, aisle-location and pricing data; verified database relationships, constraints and data integrity.	SQL, SQLite3, SQLAlchemy ORM	setup_dataset.py, database seed tables, products.py
User Authentication & Session Management	Configured password hashing and token-validation logic for secure user login, authentication and profile access.	Passlib / bcrypt, JWT, Python-Jose	auth.py, authenticated user sessions
API Specification & Architecture Documentation	Documented system requirements, API endpoints, input/output schemas and overall system architecture.	Markdown, Postman / OpenAPI (Swagger), Architecture Diagrams	architecture.md, API_REQUIREMENTS.md
Deployment Scripts & Environment Configuration	Configured local backend startup scripts, virtual environment, dependencies and cloud deployment settings.	Windows Batch Scripts, Render Configuration, Pip/Venv	start_backend.bat, render.yaml, requirements.txt

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 16.10.2026

Member 04: Jathavy Ganeshalingam

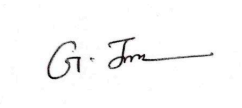
Student Name	Jathavy Ganeshalingam	**Registration No.**	D/COE/25/0006
Degree / Intake	Computer Engineering Intake - 42	**Group No.**	37
Project Title	AI-Based Smart Shopping Assistant for Budget-Aware and Personalized In-Store Shopping		

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Grocery Image Dataset Acquisition	Collected, organized and catalogued multi-angle supermarket product images under different lighting, distance and orientation conditions for AI model training.	Digital Photography, Dataset Organization, Data Curation	my_pictures/toothpaste, my_pictures/astra, my_pictures/sprite image datasets
Dataset Annotation & Verification	Prepared and verified product annotations and bounding boxes in YOLO format, and checked the training, validation and test dataset splits.	LabelImg, YOLO Annotation Format, Dataset Verification	import_dataset.py, dataset/ directory, YOLO labels
Synthetic Transaction Data Generation	Assisted in generating synthetic grocery transaction and inventory data for testing recommendation and budget-optimization functions.	Python, Random Seed Generation, JSON/CSV Data Processing	generate_3000_dataset.py, mock_data.dart
Unit & Integration Test Execution	Executed prediction, optimization and association-model test scripts, checked outputs, recorded errors and evaluated response results.	Python unittest, CLI Testing, Logging	test_prediction.py, test_optimizer.py, test execution logs

Cross-Device UI & Usability Testing	Tested the Flutter application on Android emulator and physical devices; checked responsiveness, usability, layout issues and system response.	Android Emulator, Physical Device Testing, ADB, Flutter	UI testing results, screenshots, bug reports and responsiveness verification

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 16.10.2026